

Lifelong Learning for a Kresling Origami Robot: A Plug-and-play Module for Supervised Learning of Target Adaptations

E. De Vroey

Abstract—Soft robots offer a variety of useful applications. However, the nature of their design makes them challenging to control using traditional techniques. Many applications therefore rely on machine learning-based methods, learning opaque control policies or dynamic models of the robot. This often results in overly complex and uninterpretable solutions for the problem at hand.

In this paper, a simplified approach is considered: Using a common feedforward neural network as a *target adapter* module, learned adaptations are added to the desired targets that are fed to a simple PID controller, essentially “deceiving” it into a better performance. This adapter is completely separate from the controller itself, allowing for relatively simple controllers to control and adapt to complex and evolving systems.

The approach is tested in simulation on a simple mass-spring-damper control problem, where qualitative results show near-immediate improvement in controller performance. The approach is then tested on a demonstrator origami robot, which relies for its functionality on the dynamics of the Kresling origami spring. Results show that this simple adaptation approach is robust to both changes in controller tuning and changes in system dynamics. However, the method is sensitive to the sampling frequency at which training data is recorded.

I. INTRODUCTION

Lifelong learning, the ability of a system to continuously adapt to new information throughout its lifetime, is a much sought-after feature for many applications. Rather than relying on policies that are “set in stone”, these systems learn and evolve autonomously, changing and improving their behaviors as they encounter new tasks or environments. This ability makes lifelong learning a powerful tool for systems that operate in uncertain or evolving scenarios, where changes are common or to be expected. Not in the least for robotics.

Robots, like all mechanical systems, wear down over time—joints become more compliant, motors weaken, and sensors lose precision. Traditionally, maintaining a robot’s performance might require manual software updates or recalibrations every so often (Lao et al., 2023). But as robots become more complex, especially in fields like soft robotics, where flexible, compliant materials introduce additional challenges, the need for autonomous adaptation becomes clear: By design, the materials used in the construction of soft robots are less sturdy and thus more prone to much faster deterioration, making manual updates or recalibrations impractical.

In addition, the complex system dynamics introduced by the unconventional robot configurations and use of materials make modeling their dynamics and control difficult. Deriving an analytical model for the dynamics of a soft robot can become so complicated to the point that it is impractical.

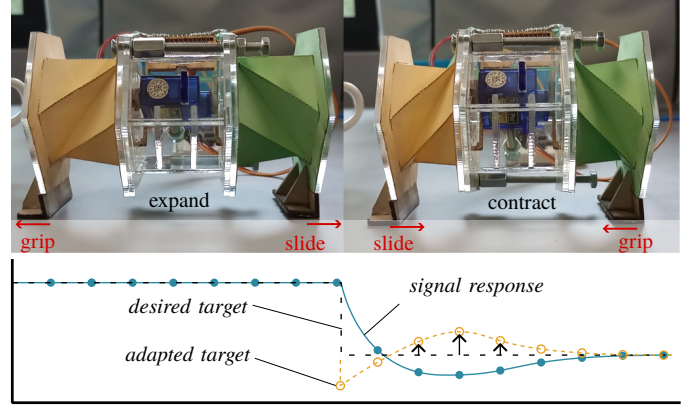


Fig. 1: The Kresling origami robot with target adaptation. *Top:* The “feet” of the robot grip when dragged backward, but slide when pushed forward, allowing the robot to move forward, also known as *two-anchor crawling* (Calisti et al., 2017). *Bottom:* The contraction target is adapted to reduce overshoot and unstable oscillations.

Soft-robotics applications therefore heavily rely on the use of machine learning (ML) methods (Bhagat et al., 2019; Kim et al., 2021). These “black-box” techniques offer the possibility of modeling the robot’s dynamics and allow for continuous adaptation but are often very opaque or complex, even though the task or environment setting of the soft robot could be something fairly simple.

This paper proposes a method of enhancing a traditional controller with ML abilities. The idea is that a basic controller can be tuned to a bare minimum sufficient performance, without worrying about the variations in dynamics introduced by soft robots. This controller is expanded with a separate module that takes care of the learning and adapting part of the challenge. The module then only needs to improve the performance of the controller, without requiring knowledge on the dynamics of the system, nor is the controller completely replaced by an uninterpretable “black-box” policy. The controller works in cooperation with the ML-module, separating the interpretable basic control from the opaque learning and adapting.

Techniques like parameter adaptation work much in the same way. An adapting or tuning module, quite often ML-based, can be “plugged” into a base controller, tuning its parameters *on the go* to achieve an optimal performance (Anaswamy & Fradkov, 2021; Landau et al., 2011). However, parameter adaptation requires knowledge on the architecture of the controller (e.g., how many, and which parameters should

be tuned), and such a module must thus be designed separately for each controller.

The proposed method tries to mitigate this downside by offering a “plug-and-play” module that can improve a controller’s performance by simply adapting the desired target it receives (Figure 1, *bottom*). In this way, no previous knowledge on the type or tuning of the controller is required. In other words, as long as the fundamental assumptions of a controller are satisfied—i.e., it requires a state and a target as input and yields a control action—the method can remain completely agnostic to the specific controller being used.

As Tang and Wei (2022) point out, with soft robots offering many advantages in miniaturization, the physical space for controllers with heavy computational power can be limited. The “plug-and-play” module offers thus offers another advantage: externalizing the heavy computational task from the soft robot, which allows a (miniaturized) robot to just run a basic controller with low computational capabilities.

As a demonstration of the technique, this target adaptation approach is tested on a demonstrator origami robot, constructed in part from paper and which relies on the widespread Kresling pattern (Kresling, 2008) for its design (Figure 1, *top*). The Kresling origami spring (KOS) is characterized by its non-linear stiffness with deflection and bistable behavior (Masana & Daqaq, 2019), making it an interesting building block for many soft-robotics applications (Masana et al., 2024). The demonstrator robot is thus a good representation of a large cross-section in soft robot designs, as well as a useful playground for testing the adaptability of the method to more complex dynamics.

II. RELATED WORK

This section will provide a useful background and context for the Kresling origami robot (Sections II-A and II-C) as well as a brief overview and comparison of related methods concerning lifelong learning (Section II-B).

A. Soft Robotics

Soft robotics is a field that has gained much interest over the past decade. Soft robots are constructed from deformable, soft materials which offer advantages in fields such as human-robot interaction, where their inherent compliance makes them safe to interact with. Other use-cases range from manipulating fragile objects, to medical treatment delivery inside the human body, and search-and-rescue (Hawkes et al., 2017; Kim et al., 2021; Ze et al., 2022).

Origami robots are a subset of soft robots. Inspired by the Japanese art of paper-folding, these robots rely on creases in their materials for both structural integrity and actuation methods. This allows for highly miniaturized designs that can self-deploy and can be actuated using affordable, lightweight mechanisms (Tang & Wei, 2022).

Of course, although the materials used often bend rather than break, they *are* prone to deteriorate faster than their rigid counterparts. Hence, combined with the aforementioned difficulty of modeling the dynamics, the usefulness of lifelong

learning in this field becomes obvious (Kim et al., 2021). Additionally, the rapid deterioration and widely available materials make soft robots a fast and economical playground for testing lifelong learning techniques as well.

B. Lifelong Learning

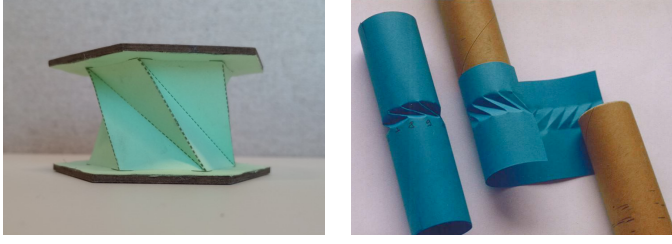
Many approaches to lifelong learning can be found in literature, with machine learning being an integral part in nearly all of them (Lesort et al., 2020). Broadly speaking, the approaches can be divided into four categories, which are not necessarily mutually exclusive: Methods that result in a “black-box” control policy, often obtained through reinforcement learning, where a policy is found by trying to optimize a *reward* function (Bhagat et al., 2019). Methods that model the dynamics of the system, to be used in combination with other controllers or to optimize an ML-based policy (e.g., Gillespie et al. (2018) and Thuruthel et al. (2019), respectively). Special architectures, such as the parameter adaptation or estimation mechanism, and iterative learning control, where a separate module capable of learning is combined with a base controller (Landau et al., 2011; R. J. Li & Han, 2005). Finally, the use of mechanisms that invoke some kind of memory, either by modifying the update method to be less destructive (e.g., *learning without forgetting* from Z. Li and Hoiem (2018)), or by implementing some form of library that can be accessed to store and retrieve information or parameters, as in e.g. Wang et al. (2020).

The techniques most similar to the method discussed in this paper are parameter adaptation/estimation and iterative learning control (ILC). In parameter adaptation, ML-based methods are used to directly tune the internal parameters of a controller. The various techniques used to achieve this rely on observing the interactions of the controller with the system or having access to the feedback signal of the controller (Annaswamy & Fradkov, 2021). Furthermore, the estimation mechanism must be designed specifically for the parameters of the controller in question. Iterative learning control on the other hand only uses the error signal to improve a controller’s performance. It achieves this by changing either the controller’s reference signal (i.e., the target) or its control action, based on the observed error during a previous trial (R. J. Li & Han, 2005). By iteratively applying this approach, the controller’s performance can be improved. However, ILC is limited to systems that are highly repetitive, i.e., the same task is repeated many times; a constraint that the proposed method will try to circumvent.

C. Kresling Origami

The Kresling origami pattern is folding-pattern used to create deployable cylindrical structures (also known as bellows) such as the one in Figure 2a. As shown in Figure 2b, the pattern is based on the creases that naturally occur when twist-buckling a cylindrical structure (Kresling, 2008).

Due to the strain induced on the folds when twisting or compressing the structure formed by the pattern, it forms a torsional and translational spring with nonlinear stiffness. The pattern can be defined by three parameters (Figure 3, *top-right*), depending on which the spring can be designed to be either mono- or bistable (Masana & Daqaq, 2019):



(a) Fully constructed 6-sided KOS. (b) Twist-buckling crease pattern. Obtained from Kresling (2008).

Fig. 2: Kresling origami pattern

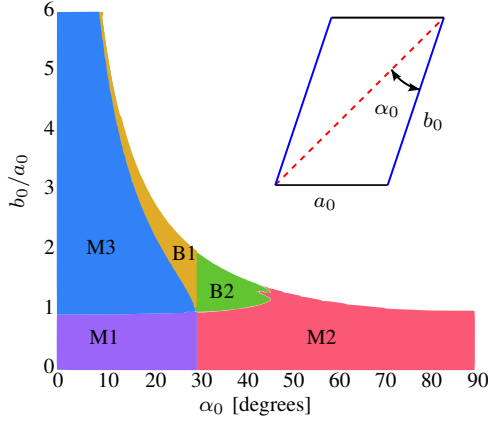


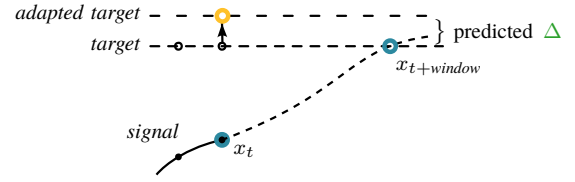
Fig. 3: Design map for a KOS with 6 sides reproduced from Masana and Daqaq (2019) and the corresponding parameters from the folding pattern, with mountain creases in blue and valley creases in dashed red.

Monostable M1, M2 and M3 type springs differ in their stable deployment height being zero, non-zero and non-zero, respectively, and their stable potential energy being non-zero, zero and non-zero, respectively. Bistable types B1 and B2 differ in their contracted stable height being non-zero and zero, respectively, and their respective potential energy being zero and non-zero. With the help of only three parameters, the map of feasible designs shown in Figure 3 can be created. Using this design map, a pattern can be designed that conforms to any of these categories. The springs used for the robot in this paper are of type B1 (??).

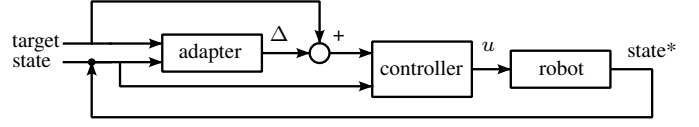
The bistable behavior of the KOS is a feature that has made the pattern popular in applications varying from mechanical switches, to aerospace structures and soft robots (Masana et al., 2024). The demonstrator robotic crawler in this paper is inspired by similar robots such as from Pagano et al. (2017) and Ze et al. (2022).

III. METHODOLOGY

The method proposed in this paper aims to provide a way of improving a given controller's performance as well as fortifying it against evolving system dynamics *without* having access to the inner workings of the controller itself. In addition, the aim is not to replace the controller entirely but to introduce an additional module to the control scheme that enables lifelong robustness to evolving system dynamics.



(a) The goal: Reach the target within the specified window. By supplying the model with the controller's target position and current state x_t , it yields a Δ that can be added to the current target.



(b) The target adaptation control scheme: The adapter is used to offset the given target before presenting it to the controller, essentially *lying* to the controller in order to improve its performance.

Fig. 4: Target adaptation

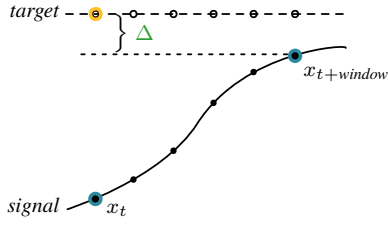
The advantage of such a method is that it could be applied to any system as a “plug-and-play” module.

The manner in which this will be achieved is by *target adaptation*. An adapter module modifies the target used by the controller to improve its performance. Consider a scenario where a controller yields a control action u to reach a certain desired target state $\mathbf{x}_d$ within a certain time window. As the system dynamics evolve, suppose this action u is now insufficient to reach the target. In response, based on the state of the system and the desired target, the adapter module “lies” to the controller about reaching $\mathbf{x}_d$ and instead provides it with $\mathbf{x}_d + \Delta$. For this adapted target, the controller now yields a larger action u^* that *does* result in the system reaching state $\mathbf{x}_d$. Figure 4a illustrates the desired outcome of the target adaptation method, the basic control scheme is presented in Figure 4b.

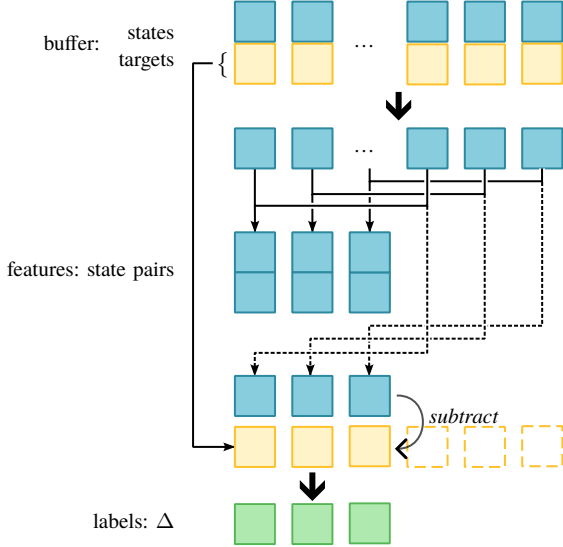
At a glance, this method resembles that of iterative learning control (ILC). However, in ILC, the adaptation to the reference signal is directly proportional to the error signal, resulting in effective behavior for a very constricted task setting, where the same task is repeated over and over. Here, the goal is to have a more universal improvement in performance, and to detach the method from the error signal.

To achieve this, the adapter is implemented as a feedforward neural network with two hidden layers with non-linear activations. To train this model to learn suitable target adaptations, interactions of the controller with the system are recorded. Specifically, a buffer stores the recorded system states with corresponding targets. The model is then provided with pairs of system states that are obtained from the buffer and are spaced by a certain time window $\mathbf{x}_t$ and $\mathbf{x}_{t+window}$. From these pairs, the model infers the target it “believes” the controller aims to reach. However, instead of learning to predict the target directly, the training labels represent the difference Δ between the target $x_{d,t}$ and the latter state in the pair (Figures 5a and 5b).

The model can be trained in a supervised manner and then deployed. During deployment, it is presented with the current state $\mathbf{x}_t$ but cannot be presented with $\mathbf{x}_{t+window}$ (since this



(a) The target adaptation problem: Based on the current position x_t and the position some prediction window later $x_{t+window}$, the model learns to predict the target the controller is trying to reach by representing it as a Δ to be added to $x_{t+window}$.



(b) The feature and label extraction process: recorded states spaced by a specified prediction window are paired up and provided as features to the adapter, which learns to predict a Δ , being the difference between the target $\mathbf{x}_{d,t}$ and state $\mathbf{x}_{t+window}$.

Fig. 5: Target adaptation during the training phase.

state is not known yet). Instead, the second state in the pair is set to the desired target. To the model, this represents a situation in which the desired target was reached from the current state within the specified time window. The idea is now that the trained model will produce a Δ that—when added to the latter state in the pair (i.e., the desired target)—results in an adapted target for which the controller will produce an improved control action (Figure 4a). In short, given a current state $\mathbf{x}_t$ and a future state $\mathbf{x}_{t+window}$, the model learned to produce the desired target. Thus, when presented with $\mathbf{x}_t$ and $\mathbf{x}_{t+window} = \mathbf{x}_d$ (supposedly reached within the time window) it provides the (adapted) target that would “push” the controller toward that result.

This approach is thus completely detached from the error signal, and allows for non-repetitive settings, as opposed to ILC. ILC, however, also provides an approach of adapting the controller’s action u directly. Trying to mimic this approach similarly to the target adaptation method described, one could train an adapter to predict a control action, or a series of actions, based on a state pair. The downside to this approach is that as the adapter improves, it would essentially replace the controller (during deployment, it receives the state and target,

and produces an action), which is one of the outcomes to be avoided, as stated earlier. Therefore, the target adaptation was considered the more elegant approach.

Finally, the reason for predicting the difference Δ between the state $\mathbf{x}_{t+window}$ and the desired target, rather than a direct prediction of the target, should also be clarified now: By designing the target adapter’s output to be the difference Δ , the model can be initialized with parameters all close to zero (essentially leaving the desired target unchanged) and then steadily improve online as interactions of the controller and system are recorded, without the need for a pretraining phase.

IV. ILLUSTRATIVE PROBLEM

To explore the proposed method, an illustrative problem is considered first. The problem consists of a simulation of a simple 1D mass-spring-damper system (Figure 6) and an imperfectly tuned PID controller. The mass is suspended in a hanging position with gravity acting on it continuously.

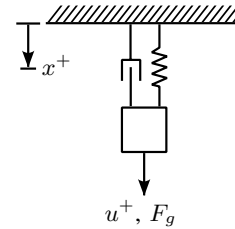


Fig. 6: Setup of the mass-spring-damper system in the illustrative problem with positive directions indicated.

The system state $\mathbf{x}$ consists of the position x and velocity $\dot{x}$. The controller aims to reach targets of the form $\mathbf{x}_d = [x_d, 0]$, but controls position only. At regular intervals, a random target position x_d is generated for the controller to reach and maintain. Each timestep, the adapter calculates the required Δ to be added to the intended target *position*, and the controller computes the control action based on the current position and the adapted target position. The system’s response to this control input is simulated and the tuple $(\mathbf{x}_0, \mathbf{x}_d)$ is added to the buffer. This buffer stores a limited number of tuples corresponding to a specified window of the most recent data: Once capacity is reached, the oldest element is removed at every timestep. At regular update intervals, the adapter is trained on the data in the buffer. To ensure meaningful training, it is important that the tuples in the training data always contain the *true* target position x_d , and not the target that was actually provided to the controller, i.e., $x_d + \Delta$. As the simulation progresses, the parameters of the dynamic system are gradually altered to simulate system degradation. The pseudo algorithm for the implementation can be found in Algorithm 1.

The simulation ran for 60s at 50Hz. The target was changed every 5s, and the adapter was trained fully online, updating every 15s. Training features were created for multiple prediction windows, this means the training features consisted of pairs $[\mathbf{x}_t, \mathbf{x}_{t+window}]$ for windows of 3, 5, 10, 15, and 25. The buffer’s budget corresponded to the past 30s of data. As a baseline, a second controller and system were also simulated, identical to the described simulation, but without the addition

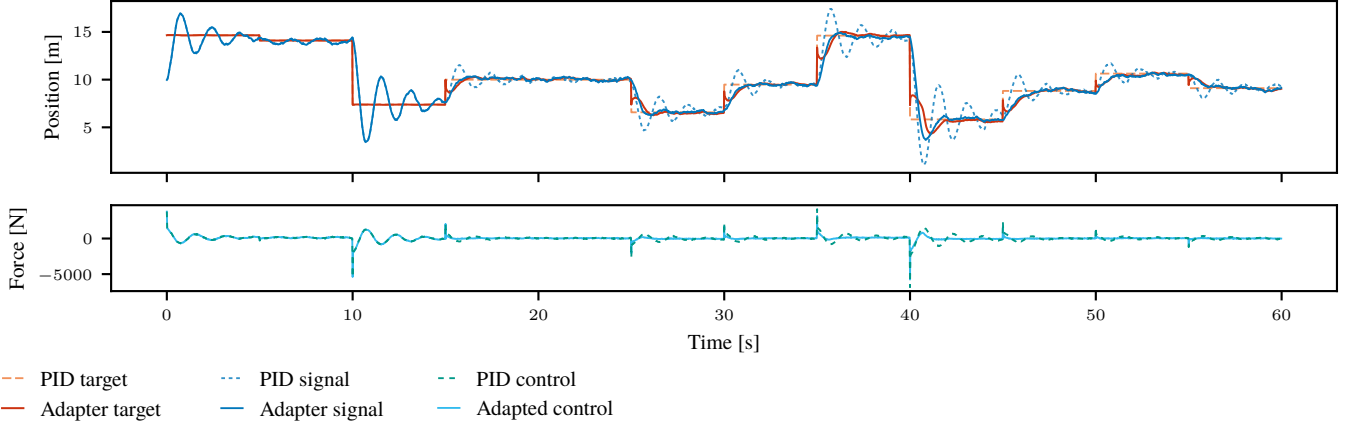


Fig. 7: The full minute simulation of the system with an aggressive controller. *Top*: The system response x and target $x_d + \Delta$ of the adapted controller, alongside the response and target of the unadapted PID controller ($\Delta = 0$). *Bottom*: The resulting adapted control inputs from the controller receiving $x_d + \Delta$ alongside the unaltered PID control inputs. During the first 15s (3 targets) the adapter has not received updates yet, and the signal is thus identical to the baseline PID signal.

Algorithm 1: Target adaptation

Data: system state $\mathbf{x}$, target $\mathbf{x}_d$, adaption Δ , control action u
Initialize model: *adapter*;
Initialize buffer: *buffer* $\leftarrow []$;
for each timestep t_i **do**
 if $\text{len}(\text{buffer}) > \text{threshold}$ **then**
 Remove the oldest element in the buffer;
 end
 if $t_i \bmod \text{target_interval} = 0$ **then**
 $\mathbf{x}_d \leftarrow \text{random_target}()$;
 end
 if $t_i \bmod \text{update_interval} = 0$ **then**
 $(\text{features}, \text{labels}) \leftarrow \text{prep_data}(\text{buffer}, \text{window})$;
 $\text{adapter.fit}(\text{features}, \text{labels})$;
 end
 $\Delta \leftarrow \text{adapter.predict}([\mathbf{x}_0, \mathbf{x}_d])$;
 $u \leftarrow \text{controller.compute_control}(\mathbf{x}_0, \mathbf{x}_d + [\Delta, 0])$;
 Simulate the system $\mathbf{x} \leftarrow \text{response}(\mathbf{x}_0, u)$;
 Update system parameters;
 Append $(\mathbf{x}_0, \mathbf{x}_d)$ to *buffer*;
 $\mathbf{x}_0 \leftarrow \mathbf{x}$;
end

of an adapter. To both the baseline and adapted signal, a small amount of Gaussian noise was added to simulate measurement noise.

A. An Over-Tuned Controller

For a first simulation, a combination of a system and controller was chosen where the controller's gains were tuned too high. In Figure 7, the result of the simulation can be observed. At the start, due to the near-zero initialization of the adapter's parameters, the signal response of the system controlled by the adapted controller is indistinguishable from that of the baseline PID controller. However, as soon as the

adapter has trained on the buffer, the changes to the target become visible.

The effects of the adaptation can be more clearly observed in Figure 8. Since the baseline PID controller is too aggressive, the signal overshoots and oscillates significantly before settling. In contrast, the adapter counters this by bringing the target closer to the current state. This is the logical result of the buffer containing data corresponding to the overshoots, where the adapter would often encounter training features for which the labeled target position¹ x_d is somewhere in between x_t and $x_{t+\text{window}}$. Therefore, upon receiving the input feature $[\mathbf{x}_t, \mathbf{x}_d]$ it produces an adapted target position in between x_t and x_d . As the state approaches the adapted target, the adapter starts producing targets closer to the reference target, guiding the system towards this desired state.

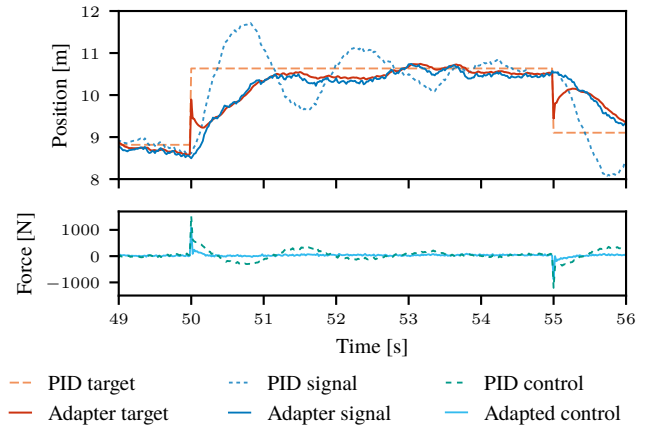


Fig. 8: A close-up of the second to last target in the one-minute simulation. The effects of the adapter can be more clearly observed: The overshoot (seen in the baseline PID signal) is countered by bringing the target closer to the current state.

¹Rather, the labeled $\Delta = x_d - x_{t+\text{window}}$, but the explanation provided is equivalent and somewhat clearer.

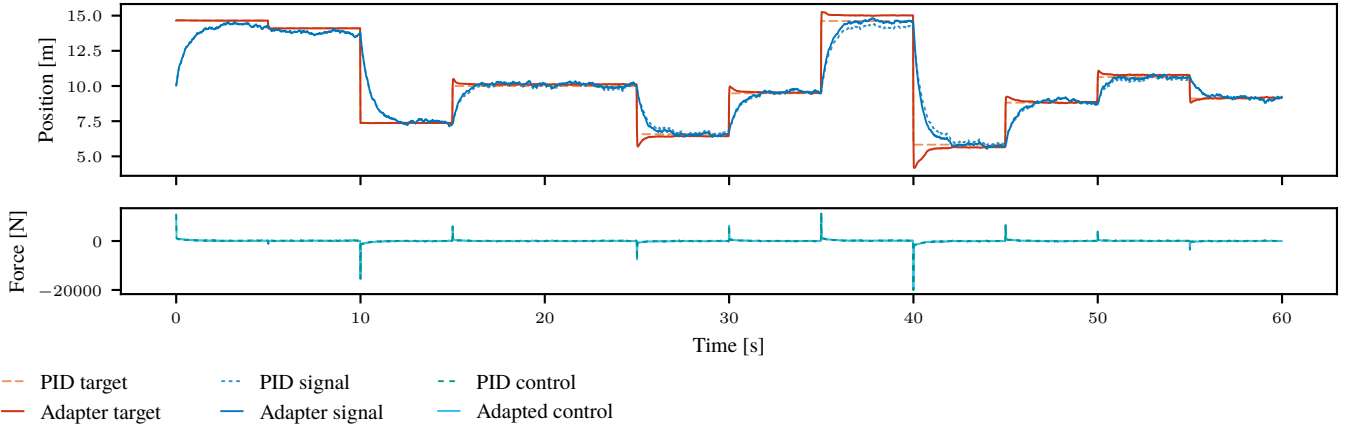


Fig. 9: The full minute simulation of the system with an under-responsive controller. *Top*: The system response and target of the adapted controller, alongside the response and target of the unadapted baseline PID controller. *Bottom*: The resulting adapted control inputs from the controller receiving the altered target alongside the unaltered baseline PID control inputs.

B. An Under-Tuned Controller

For a second simulation, the system and controller were designed such that the controller was more conservative or *under-tuned*. Additionally, the signal produced by the baseline PID controller has a clear steady-state error. The result of the simulation can be found in Figure 9.

Once more, the adapter is able to improve the controller's performance after just a single update. In Figure 10 it can be seen that the adapted target is adjusted away from the current state. This is because training features created from the data in the buffer frequently include state pairs $\mathbf{x}_t, \mathbf{x}_{t+window}$ where the target position x_d is positioned further from x_t than it is from $x_{t+window}$. As a result, the adapter learns to predict adaptations that would move x_d farther from x_t , making the controller more aggressive. Another effect of the adapter is the consistent offset applied to the target: In the training features, the adapter regularly encounters pairs $\mathbf{x}_t, \mathbf{x}_{t+window} = \mathbf{x}_t$, which appear to represent the target state being reached, even though $x_{t+window} \neq x_d$. These features are the result of the steady-state error present in the signal, leading the adapter to learn to apply an offset to the target to counteract this.

C. Preliminary Conclusions

These qualitative results show that the adapter is able to improve controller's behavior in a simple *simulated* dynamic system. The adapter is also making target adaptations that are intuitive and logical, given the specific settings of the system and controller. The simulations thus provide a good foundation that the method is at least reasonable. For a real-world test of the method, a demonstrator robot is required (Section V) and a good test setup with a comprehensive set of metrics to quantify the results (Section VI).

V. A KRESLING ORIGAMI ROBOT

A. Mechanical Details

As mentioned in Section II-A, origami robots—and by extension, soft robots—are a suitable use-case for lifelong

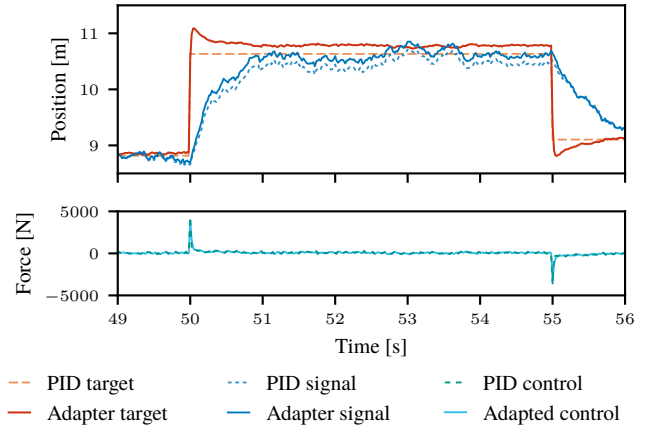
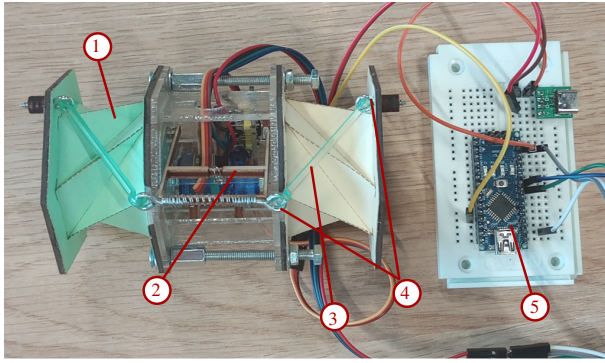


Fig. 10: A close-up of the second to last target in the one-minute simulation. The target is initially moved further from the current state to 'pull' it towards the desired state in a shorter period. In addition, the adapted target is offset from the desired state as to counter the steady-state error seen in the baseline PID signal.

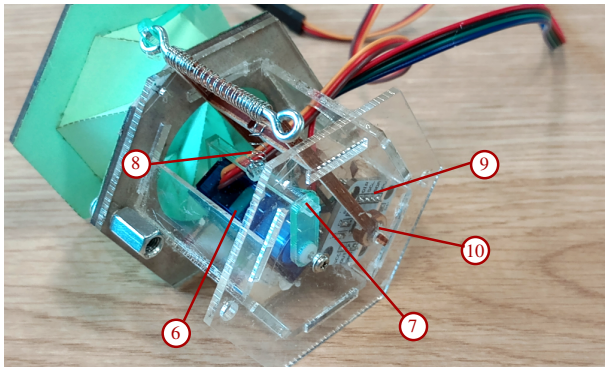
learning techniques. To evaluate the method described in Section III, a robot was designed that serves as a good analogue to the simulated problem, whilst also using Kresling origami as a key part of its functionality. Figure 11 provides important nomenclature for reference.

The robot is a simple crawler consisting of two Kresling origami springs (KOS) (Figure 11a, item 1) that are mirror images of each other, connected in series, with the actuation mechanism in between. The KOSs in question are of the bistable B1 type (Section II-C) with perforated folds to facilitate actuation (refer to ?? for design details and templates). Both springs can expand and contract in sync, producing a crawling-like motion when combined with specially designed "feet" (see Figure 1 in Section I). To demonstrate target adaptation, however, the movement of the robot as a whole is not very important. Rather, the state of the KOSs is the



- | | |
|--------------------|--------------|
| 1) KOS | 4) Eye screw |
| 2) Actuation arm | 5) Arduino |
| 3) Connecting link | |

(a) The Kresling origami robot.



- | | |
|-----------------|-----------------|
| 6) Servo motor | 9) Angle sensor |
| 7) Servo arm | 10) Magnet |
| 8) Metal spring | |

(b) Actuation mechanism

Fig. 11: Annotated views of the robot and its most important actuation mechanisms.

state of interest that will be controlled.

A KOS can be actuated both by linear motion (pushing or pulling on the spring) and by rotational motion (twisting). Since the two motions are directly related (i.e. twist-buckling, discussed in Section II-C), a contracting or expanding motion can be measured as the angle of the KOS's twist. The twist is induced by turning a rotating arm (2) which is connected to the ends of the KOSs by two links (3) that hook into eye-screws (4). To turn the arm, a micro servo motor (6) is employed. In the illustrative problem, the controller yields a force to exert on the system in order to reach the target position. However, most micro servo's come with a near perfect position controller built-in, which cannot be accessed or changed. Therefore, to bridge the gap between the simulation and the real-world implementation, the position output of the servo is converted to a force output by the use of a small, metal spring (8). The servo then does not directly act on the actuation arm, but rotates a servo arm (7) instead, which pulls the spring and thereby exerts a force on the actuation arm. As previously mentioned, the state of the KOS can be measured as the angle of twist, since

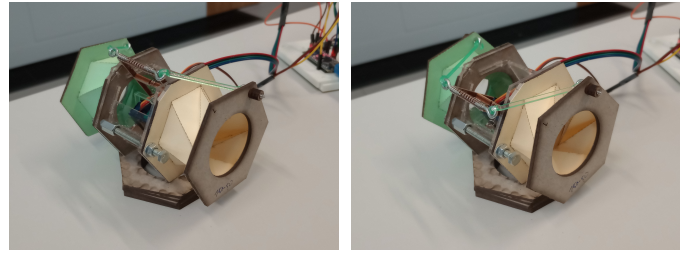


Fig. 12: *Left*: The robot at its fully extended state, corresponding to the “home” position of the servo and actuation arm. *Right*: The fully contracted robot, corresponding to the maximum (capped) servo output, near the mechanical limit of the actuation arm.

it is directly related to its contraction. Likewise, assuming perfectly rigid links with negligible clearance between the hooks and eye-screws, the angle of twist is directly related to the angle of the actuation arm. In the full implementation, the robot's state $\mathbf{x}_t$ will thus be measured as the actuation arm's angular position and velocity $[\theta, \dot{\theta}]$. This angle is measured using a contactless sensor (9) that can measure the on-axis rotation of a small magnet (10) attached to the actuation arm. The sensor relays this information to an Arduino Nano (5) board, which runs a PID controller that returns the control action, i.e., an angular position for the servo. Furthermore, there are mechanical limits for the rotation of both the actuation arm and servo arm, imposed by the edge of the container housing the actuation mechanism and the supports holding the servo motor, respectively (Figure 12, *right*). However, the design is such that the servo's mechanical limit corresponds to a θ below the actuation arm's limit.

B. Software Implementation

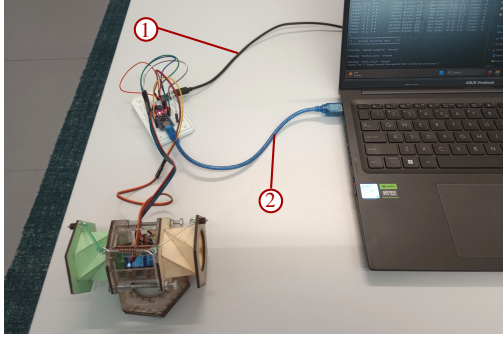
The software implementation consists of two main components: The nominal control of the robot, and the target adaptation.

The Arduino board is responsible for the nominal control, implemented as a simple PID controller. Additionally, it takes care of calibrating the measurements upon startup, i.e. (re)setting the actuation and servo arm to their minimum “home” position (Figure 12, *left*).

Target adaptation is implemented on an external computer. Through a serial connection (Figure 13, item 2) with the Arduino, it both receives the measurements to be stored in the buffer, and sends the adapted targets to be used by the PID controller. The main process was summarized in Section IV by Algorithm 1. However, on the computer, the several steps are divided amongst separate modules that act in parallel so that real-time performance is ensured. A more detailed description of the software implementation can be found in ??.

VI. EXPERIMENTAL SETUP

For each experiment, a minimum of two sets of KOSs was required: One set serving as a baseline, with the Arduino receiving unadapted targets, and the second set used for testing target adaptation. All experiments used a 60s buffer and 15s



- 1) Servo motor power supply
- 2) Serial connection and Arduino power supply

Fig. 13: The test setup.

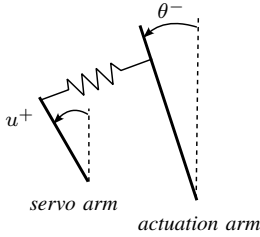


Fig. 14: Nominal signs of the angles for the servo and actuation arm during operation.

intervals between adapter updates, with the first update at 60s. The adapter trained on prediction windows of 3, 5, 10, 15, and 25 steps. The *true* target, with values depending on the experiment, was updated every 5s, following a predefined random sequence. This sequence amounts to 5min of deployment time and is repeated according to the full experiment duration. The signals corresponding to each 5min sequence are recorded and saved for evaluation. This way, the evolution of the performance metrics can be studied, and a fair comparison between the baseline PID controller and adapted controller is ensured. Figure 13 shows the general experimental setup.

A. Sign Conventions and Unit Clarification

Before the experiments and results can be properly understood, it is necessary to clarify certain sign and unit conventions.

For example, the contactless magnetic sensor denotes a clockwise rotation (as viewed from the top) w.r.t. the datum as a positive angle, and likewise, a counterclockwise rotation as a decrement in the angle (Rob Tillaart, 2024). In practice, this means that a positive control input from the servo (i.e. a positive angle) results in a decrement in θ . Because the actuation arm's calibration point is set when the servo is at its minimum output, the operational range of the actuation arm is entirely in the negative domain. Figure 14 illustrates the servo arm and actuation arm in a configuration representative of a typical scenario during the experiments, with the signs of the angles indicated.

In addition, The *Servo* Arduino library uses the `writeMicroseconds(value)` function to write to the

`servo.writeMicroseconds(value)` maps a servo's range to a value between 1000 and 2000 μ s (depending on the servo's manufacturer), relating to the pulse width in microseconds of the signal that the Arduino sends to the servo motor (Arduino, 2024).

Finally, during the experiments, the Arduino initialized a *count* value to include in the tuple sent to the computer. This counter starts at 0 and simply increments by 1 with each iteration of the Arduino's loop. Since the interactions between the Arduino and computer and the several threads on the computer are independent of timing (i.e., the modules do not have to "wait for each other"), the counter's value is essential to accurately plot the several signals recorded during the experiments.

B. Performance Metrics

The performance metrics used to evaluate the 5min sequences are the mean rise time (MRT), mean settling time (MST), mean overshoot (MO), mean absolute error (MAE), mean absolute velocity (MAV), and normalized total variance (NTV).

The mean rise time is calculated as the mean of all the rise times corresponding to each target in the 5min sequence. The rise time is defined as the time it takes the robot to come within 10% of the target state, as calculated from the initial position.

Similarly, the settling time is the average of the settling times for each target. The robot state is considered settled when it remains within 5% of the target state, as calculated from the initial position.

The mean overshoot is the average of the maximum overshoots observed for each target.

The mean absolute error represents the average difference between the robot's angle θ and the true target θ_d .

The mean absolute velocity refers to the measurements of $\dot{\theta}$. This metric is meant to provide a measure for the "chaoticness" of the robot's signal response.

The normalized total variance is calculated as the sum of the absolute of all increments and decrements in the control action, normalized over time². This aims to quantify the controller's effort.

REFERENCES

- Annaswamy, A. M., & Fradkov, A. L. (2021). A Historical Perspective of Adaptive Control and Learning. *Annual Reviews in Control*, 52, 18–41. <https://doi.org/10.1016/j.arcontrol.2021.10.014>
- Arduino. (2024). Servo Reference. Retrieved September 24, 2024, from <https://www.arduino.cc/reference/en/libraries/servo/writemicroseconds/>
- Bhagat, S., Banerjee, H., Tse, Z. T. H., & Ren, H. (2019, January). Deep reinforcement learning for soft, flexible robots: Brief review with impending challenges. <https://doi.org/10.3390/robotics8010004>
- Meta-studie.

²Actually, it is normalized over the *count*, see Section VI-A.

- Calisti, M., Picardi, G., & Laschi, C. (2017, May). Fundamentals of soft robot locomotion. <https://doi.org/10.1098/rsif.2017.0101>
- Gillespie, M. T., Best, C. M., Townsend, E. C., Wingate, D., & Killpack, M. D. (2018). Learning nonlinear dynamic models of soft robots for model predictive control with neural networks. *2018 IEEE International Conference on Soft Robotics (RoboSoft)*, 39–45. <https://doi.org/10.1109/ROBOSOFT.2018.8404894>
- Hawkes, E. W., Blumenschein, L. H., Greer, J. D., & Okamura, A. M. (2017). A soft robot that navigates its environment through growth. *Science Robotics*, 2(8), 1–8. <https://doi.org/10.1126/scirobotics.aan3028>
- Kim, D., Kim, S.-H., Kim, T., Kang, B. B., Lee, M., Park, W., Ku, S., Kim, D., Kwon, J., Lee, H., Bae, J., Park, Y.-L., Cho, K.-J., & Jo, S. (2021). Review of machine learning methods in soft robotics (G. Gu, Ed.). *PLOS ONE*, 16(2), e0246102. <https://doi.org/10.1371/journal.pone.0246102>
- Kresling, B. (2008). Natural twist buckling in shells: from the hawkmoth's bellows to the deployable Kresling-pattern and cylindrical Miura-ori. *Proceedings of the 6th International Conference on Computation of Shell and Spatial Structures IASS-IACM 2008: "Spanning Nano to Mega"*, (May), 1–4.
- Landau, I. D., Lozano, R., M'Saad, M., & Karimi, A. (2011). *Adaptive Control*. Springer London. <https://doi.org/10.1007/978-0-85729-664-1>
- Lao, D., Quan, Y., Wang, F., & Liu, Y. (2023). Error Modeling and Parameter Calibration Method for Industrial Robots Based on 6-DOF Position and Orientation. *Applied Sciences (Switzerland)*, 13(19), 10901. <https://doi.org/10.3390/app131910901>
- Lesort, T., Lomonaco, V., Stoian, A., Maltoni, D., Filliat, D., & Díaz-Rodríguez, N. (2020). Continual learning for robotics: Definition, framework, learning strategies, opportunities and challenges. *Information Fusion*, 58, 52–68. <https://doi.org/10.1016/j.inffus.2019.12.004>
- Li, R. J., & Han, Z. Z. (2005). Survey of iterative learning control. *Kongzhi yu Juece/Control and Decision*, 20(9), 961–966. <https://doi.org/10.1109/mcs.2006.1636313>
- Li, Z., & Hoiem, D. (2018). Learning without Forgetting. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 40(12), 2935–2947. <https://doi.org/10.1109/TPAMI.2017.2773081>
- Masana, R., Dalaq, A. S., Khazaaleh, S., & Daqaq, M. F. (2024). The Kresling Origami Spring : A Review and Assessment. *Smart Materials and Structures*, 33. <https://doi.org/10.1088/1361-665X/AD2F6F>
- Masana, R., & Daqaq, M. F. (2019). Equilibria and bifurcations of a foldable paper-based spring inspired by Kresling-pattern origami. *Physical Review E*, 100(6), 063001. <https://doi.org/10.1103/PhysRevE.100.063001>
- Pagano, A., Yan, T., Chien, B., Wissa, A., & Tawfick, S. (2017). A crawling robot driven by multi-stable origami. *Smart Materials and Structures*, 26(9). <https://doi.org/10.1088/1361-665X/aa721e>
- Rob Tillaart. (2024). Arduino library for AS5600 magnetic rotation meter. Retrieved September 24, 2024, from <https://github.com/RobTillaart/AS5600>
- Tang, J., & Wei, F. (2022). Miniaturized Origami Robots: Actuation Approaches and Potential Applications. *Macromolecular Materials and Engineering*, 307(2), 2100671. <https://doi.org/10.1002/mame.202100671>
- Thuruthel, T. G., Falotico, E., Renda, F., & Laschi, C. (2019). Model-Based Reinforcement Learning for Closed-Loop Dynamic Control of Soft Robotic Manipulators. *IEEE Transactions on Robotics*, 35(1), 124–134. <https://doi.org/10.1109/TRO.2018.2878318>
- Wang, Z., Chen, C., & Dong, D. (2020). Lifelong Incremental Reinforcement Learning with Online Bayesian Inference. *IEEE Transactions on Neural Networks and Learning Systems*, 33(8), 4003–4016. <https://doi.org/10.1109/TNNLS.2021.3055499>
- Ze, Q., Wu, S., Nishikawa, J., Dai, J., Sun, Y., Leanza, S., Zemelka, C., Novelino, L. S., Paulino, G. H., & Zhao, R. R. (2022). Soft robotic origami crawler. *Science Advances*, 8(13), 7834. <https://doi.org/10.1126/sciadv.abm7834>